



DFNS ARCHITECTURE · V1.0 · ONCHAIN CORE BANKING

The Governance Engine

Also called the Integrity Framework

How DFNS commits, validates, and signs the most security-critical custody state — users, credentials, and signing requests — with a customer-controlled engine that can run on untrusted infrastructure.



SECURITY MODEL

TRUST BOUNDARY

MERKLE COMMITMENT

INTENT VERIFICATION

ROOT SIGNING

Three parts, from trust model to operations

PART I

The trust model

Why the state database is critical, the invariant we rely on, and the two components split across the trust boundary.

PART II

The cryptographic core

The Merkle commitment, why forgery and rewind fail, and how every intent is verified before a signature.

PART III

Keys, recovery & scope

The engine key, state store and disaster recovery, onboarding a live org, and exactly what is defended.

PART I

The trust model

What is critical, the single invariant we lean on, and the two components on opposite sides of the boundary.

The state database is as critical as the signing keys

A custody platform trusts a database to record which credential belongs to which user, and whether it is active. That record authorizes withdrawals — so an attacker who can write to it never needs to steal a key.

The attack we remove

Insert a credential, or reactivate a revoked one, directly in the database — and authorize a signature that the customer never approved.

What the engine changes

It removes the trust placed in that database. Forged or rewound state is rejected at signing time, not silently honored.

02 — THE CORE INVARIANT

01

The authoritative signed root is held by a **customer-controlled engine** — which persists only that root hash, never a tree the untrusted side could reconstruct and rewind.

The whole security argument reduces to this one invariant. Hold it, and the untrusted side has nothing it can forge from.

One system, two components on opposite sides of the boundary

	Proxy component UNTRUSTED	Engine component TRUSTED
Runs on	DFNS infrastructure	Customer premises
Its job	Polls the platform databases, builds Merkle proofs, packages the customer's signed intent, calls the engine, and mirrors validated results back.	Validates each intent, verifies the proof against its own root, mutates the tree, signs the new root, and signs transaction payloads.
Holds signing key	No — it cannot forge a valid root	Yes — software seed or HSM
Persistent state	Governance DB — the full Merkle tree and audit trail (Postgres)	The 32-byte root hash and the wrapped key only (SQLite)

Requests flow one way, across the boundary



Application requests always flow proxy → engine. The engine polls nothing and initiates no business logic of its own.

The engine dials out — so the customer opens no inbound ports

The two components speak mutual-TLS gRPC. The direction is deliberately inverted for on-premises deployment: the engine connects as a TLS client, then serves the governance service back over that single authenticated connection.

No inbound firewall ports

The customer premises accept no incoming connections. The engine initiates the link outward; DFNS never reaches in.

A handshake that reconciles first

Each connection opens by agreeing on the engine's current persisted root and mode — so divergence surfaces at connect time, not during a later mutation.

PART II

The cryptographic core

The Merkle commitment, why the untrusted side cannot forge or rewind, and how every intent is verified before a signature is produced.

One 32-byte root anchors all governed state

HASH

SHA-256 with domain separation — leaves and internal nodes hash under distinct prefixes, so a leaf can never be reinterpreted as a node.

STRUCTURE

Binary tree with breadth-first indexing ($\text{root} = 0$, children $2i+1 / 2i+2$). It grows and reindexes on resize; capacity is not fixed.

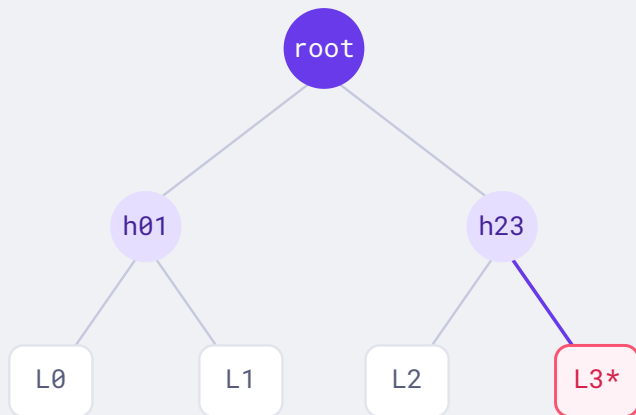
LEAF CONTENT

A version-scoped projection of the source record — a field allow-list — plus engine-injected integrity fields such as `credCounter`.

OPERATIONS

Inclusion check, add-leaf, and update-leaf drive governance. Revocation is a soft-delete — leaves are never physically removed.

Change any leaf, and the signed root changes



A tampered leaf (**L3***) propagates a new hash all the way to the root. The engine holds only the root it signed — so the altered tree yields a root it never signed.

A proof that doesn't match is returned as a **precondition failure**. The organization's queue halts — it is never silently retried.

04 — THE CONSEQUENCE

The untrusted proxy cannot present a proof the engine's root will accept for state it did not itself commit.

Cannot forge

Cannot rewind

Cannot replay

Every mutation is authorized by a proven credential

1

Prove the signer

The signing credential's leaf must be included in the current root.

2

Verify the signature

Passkey: over `authData` ||
`SHA-256(clientData).Key`
credential: over raw
`clientData`.

3

Bind the challenge

Require `challenge.path ==`
`actionPath` and
`payloadHash == SHA-`
`256(payload)`.

4

Data-aware checks

Confirm signer and user are active, and identity fields such as email match the signed payload.

Run entirely on the trusted side — no mutation and no signature happen before all four pass.

The scheme comes from the proven key, not the request

The signature scheme is derived from the credential's Merkle-proven public key — never from a request-supplied algorithm string. An attacker on the untrusted side cannot downgrade or substitute it.

Supported schemes, accepted per credential

RSA

ECDSA P-256

P-384

P-521

secp256k1

Ed25519

credCounter — a counter that lives inside the signed leaf

A user's first credentials are added during registration without a signed intent. To stop an attacker injecting more through that same unsigned path, the engine tracks a per-user counter — and once it is non-zero, the unsigned path is closed.

Because the counter is part of the signed leaf, it can't be altered without producing a root the engine did not sign.

```
# no-intent add_credential
if intent absent and
  credCounter != 0:
  reject

active Fido2 / Key → +1
deactivate → -1
RecoveryKey → no change
```

PART III

Keys, recovery & scope

The engine key as the customer's root of trust, the state store and disaster recovery, onboarding an existing org, and exactly what is defended.

The engine key is the customer's root of trust

SCHEME

Ed25519, 64-byte signatures — chosen over ECDSA so the signed input can be a raw, arbitrary-length message (e.g. Solana's non-prehashed payloads).

PROTECTION

In production, an HSM generates the private key inside the module — it never leaves, persisted as a wrapped blob. A software-seed mode exists for testing only.

The engine signs the root after every mutation, and signs each transaction payload in the read-only flow — with the same key.

The trusted side stores almost nothing

State store

- Current 32-byte root hash
- The wrapped engine key
- Nothing more — the full tree lives on the untrusted side

SQLite

Disaster recovery

Optional mTLS replication of the root and wrapped key to a backup server, with streaming liveness monitoring.

TOFU turns governance on for a live organization

A customer-initiated, one-shot bootstrap that snapshots an organization's current users and credentials into the tree — so governance can be enabled for an existing org, not only a new one.

Customer-gated

Enabled by a customer-set flag that the untrusted side cannot switch on.

Safe ordering

Adds credentials recovery-keys-first, so `credCounter` lands correctly.

Reconcilable

Emits an audit log the customer checks against a pre-flight manifest before confirming.

What the framework defends against



Compromised DFNS infrastructure

Forging or altering governed records — rejected at proof verification.



Rewind or replay of state

The engine's persisted root is the only authority.



Unsigned credential injection

Outside registration — blocked by credCounter.



Scheme downgrade

The scheme is bound to the Merkle-proven key.

What is out of scope, by design

01 The wallet signing keys themselves

That is the HSM signing plane — covered in a separate document.

02 Availability of DFNS infrastructure

This framework governs integrity, not uptime.

03 Compromise of the customer's engine host or key

That host is the trust anchor — protecting it is the customer's responsibility.

NET EFFECT

A mathematically verifiable guarantee that the identity and authorization data behind every signature is exactly what the customer approved.

Enforced by the customer

A component they operate and a key they hold.

Fails closed

Any tampering halts the queue rather than being silently honored.

Same model, extending

Policy and permission governance follow the same signed commitment.

